



Docker Commands & Interview Handbook

Beginner Guide with Commands, Examples & Interview Questions — Learn Docker from Zero with Simple Language

WHAT YOU WILL LEARN

- ▶ What is Docker & Why Use It
- ▶ Core Docker Concepts
- ▶ 21 Essential Docker Commands
- ▶ Command Cheat Sheet
- ▶ Practical Beginner Examples
- ▶ Common Errors & Fixes
- ▶ Best Practices
- ▶ 30 Interview Questions

By the end of this guide, you will understand Docker clearly enough to use it in real projects and confidently answer interview questions.

Beginner Friendly · Simple English · Interview Ready · Practical Examples

Created By : **Kartik Mungole** · @kartikmungole

Docker Fundamentals

Docker helps you package your application with everything it needs — code, libraries, settings — into a small box called a **container**. This box runs the same way everywhere, on any computer.

What is Docker?

Docker is a tool that lets developers put their application inside a container. Think of a container like a lunchbox — it has everything your food (application) needs: the food itself, cutlery, napkins. You can carry this lunchbox anywhere and eat the same meal.

Before Docker, developers often said *"It works on my machine!"* — Docker fixes this problem. Your container works exactly the same on every machine.

Why is Docker Used?



Fast Deployment

Start your app in seconds. No complex setup required on new machines.



Consistency

Same container runs identically on developer laptop, test server, production.



Isolation

Each app lives in its own container. They don't interfere with each other.



Portability

Run anywhere — Linux, Windows, Mac, cloud servers, anywhere.

Docker vs Virtual Machines

Both Docker and Virtual Machines (VMs) help run apps in isolation. But Docker is much lighter and faster.

Feature	Docker Container	Virtual Machine
Startup Time	Seconds	Minutes
Size	MBs (very small)	GBs (very large)
Uses OS kernel	Shares host OS	Has its own OS
Performance	Near-native speed	Slower due to overhead
Resource Usage	Very low	High

🟡 **Simple Analogy:** A VM is like renting a whole apartment. Docker is like renting just one room. You use only what you need — much cheaper and faster.

Important Docker Concepts

Before using Docker commands, understand these key terms. Every Docker user must know these. Don't skip this section — it makes everything else much easier.

Docker Image READ-ONLY

A blueprint or template to create containers. Like a recipe — you follow it to cook (run) a container. Images don't run by themselves.

```
docker pull nginx
```

Docker Container RUNNING

A live, running instance of an image. Like a cake baked from a recipe. You can have many containers from one image.

```
docker run nginx
```

Dockerfile BUILD FILE

A text file with instructions to build a custom Docker image. You write steps like: use this base image, copy files, run commands.

```
FROM node:18
```

Docker Hub REGISTRY

A public library of Docker images (like GitHub for code). Download ready-made images for nginx, mysql, ubuntu, node, python and more.

```
hub.docker.com
```

Volumes STORAGE

Persistent storage for containers. When a container stops, its data is lost — unless you use a volume. Volumes keep data safe.

```
docker volume create mydata
```

Networks CONNECT

Docker networks let containers talk to each other. Like putting containers on the same Wi-Fi — they can communicate privately.

```
docker network create mynet
```

Port Mapping ACCESS

Opens a door from your computer to the container. Container runs on port 80 inside, you access it on port 8080 on your machine.

```
docker run -p 8080:80 nginx
```

Tags VERSION

Labels that identify different versions of an image. Like version numbers.

```
nginx:latest
```

vs

```
nginx:1.25
```

```
docker pull node:18-alpine
```

The Docker Build Process

How Docker works in 4 simple steps:

1. Write Dockerfile → 2. Build Image (docker build) → 3. Run Container (docker run) → 4. App is Live 🎉

Essential Docker Commands

These are the most important Docker commands you will use daily. Learn each one — understand what it does and when to use it.

Command	What It Does	Example
<code>docker --version</code>	Check which Docker version is installed on your machine	<code>Docker version 24.0.5</code>
<code>docker pull <image></code>	Download an image from Docker Hub to your machine	<code>docker pull nginx</code>
<code>docker images</code>	List all Docker images downloaded on your machine	<code>docker images</code>
<code>docker ps</code>	Show only currently running containers	<code>docker ps</code>
<code>docker ps -a</code>	Show ALL containers including stopped ones	<code>docker ps -a</code>
<code>docker run <image></code>	Create and start a new container from an image	<code>docker run nginx</code>
<code>docker stop <id></code>	Gracefully stop a running container	<code>docker stop abc123</code>
<code>docker start <id></code>	Start a stopped container again	<code>docker start abc123</code>
<code>docker restart <id></code>	Stop and start a container immediately	<code>docker restart abc123</code>
<code>docker rm <id></code>	Delete a stopped container permanently	<code>docker rm abc123</code>
<code>docker rmi <image></code>	Delete a Docker image from your machine	<code>docker rmi nginx</code>
<code>docker exec -it <id> bash</code>	Open a terminal inside a running container	<code>docker exec -it web bash</code>
<code>docker logs <id></code>	View the output logs of a container	<code>docker logs web</code>
<code>docker build -t <name> .</code>	Build a Docker image from your Dockerfile	<code>docker build -t myapp .</code>
<code>docker inspect <id></code>	Show detailed info about a container or image	<code>docker inspect web</code>
<code>docker stats</code>	Live CPU, memory, network usage of running containers	<code>docker stats</code>
<code>docker system prune</code>	Remove all stopped containers, unused images to free space	<code>docker system prune</code>
<code>docker volume ls</code>	List all Docker volumes on your machine	<code>docker volume ls</code>
<code>docker network ls</code>	List all Docker networks available	<code>docker network ls</code>
<code>docker-compose up</code>	Start all services defined in docker-compose.yml	<code>docker-compose up -d</code>
<code>docker-compose down</code>	Stop and remove all services from docker-compose	<code>docker-compose down</code>

Practical Beginner Examples

The best way to learn Docker is by doing. Follow these examples step by step. Each example builds your confidence and understanding.

Example 1 — Run nginx Web Server

nginx is a popular web server. Let's run it inside a Docker container in just one command.

```
# Pull and run nginx, map port 8080 on your machine to 80 inside container docker run -d -p 8080:80 --name myweb nginx # Open browser and visit: http://localhost:8080 # You will see the nginx welcome page — your web server is live! # Check it is running docker ps CONTAINER ID NAME STATUS PORTS a1b2c3d4e5f6 myweb Up 2 seconds 0.0.0.0:8080->80/tcp
```

💡 **-d** means detached mode — container runs in background so your terminal is free. **-p 8080:80** means: my port 8080 → container port 80.

Example 2 — Enter a Running Container

Sometimes you need to go inside a container to check files or debug. Use **exec**.

```
# Open bash terminal inside running container "myweb" docker exec -it myweb bash # Now you are INSIDE the container! You can run Linux commands root@a1b2c3d4e5f6:/# ls bin boot dev etc home lib usr var ... # Type 'exit' to come back out root@a1b2c3d4e5f6:/# exit
```

Example 3 — Create a Simple Dockerfile

Build your own custom Docker image for a Node.js app. Create a file named **Dockerfile** (no extension) with this content:

```
# Step 1: Start from official Node.js image FROM node:18-alpine # Step 2: Set the working directory inside the container WORKDIR /app # Step 3: Copy your package.json and install dependencies COPY package.json . RUN npm install # Step 4: Copy all your app files COPY . . # Step 5: Tell Docker which port the app uses EXPOSE 3000 # Step 6: Command to start the app CMD ["node", "app.js"]
```

```
# Now build your image from this Dockerfile docker build -t my-node-app . # Run your custom image docker run -d -p 3000:3000 my-node-app # Visit http://localhost:3000 — your app is running!
```

💡 **Beginner Note:** The **.** at the end of docker build means "use Dockerfile in current folder". Never forget the dot!

Common Errors + Best Practices

Common Docker Errors & How to Fix Them

Error: Bind for 0.0.0.0:8080 failed – port is already allocated

Why: Another app or container is already using port 8080.

✅ **Fix:** Use a different port — try `-p 9090:80` instead.

Error: Unable to find image 'myimage:latest' locally

Why: Wrong image name, typo, or image not yet downloaded.

✅ **Fix:** Run `docker pull myimage` first. Check Docker Hub for the correct name.

Error: Permission denied while trying to connect to Docker daemon

Why: Your user does not have permission to run Docker on Linux.

✅ **Fix:** `sudo usermod -aG docker $USER` then log out and back in.

Error: Container exits immediately after starting

Why: The main process inside the container finished with nothing to keep it alive.

✅ **Fix:** Run `docker logs <id>` to see error. For ubuntu: `docker run -it ubuntu bash`

Error: Cannot connect to Docker daemon – is it running?

Why: Docker service has not been started on your machine.

✅ **Fix:** Open Docker Desktop (Windows/Mac) or run: `sudo systemctl start docker`

Docker Best Practices

1

Always use official images

Use images from Docker Hub official sources like nginx, node, python. They are tested, secure and maintained.

2

Use lightweight base images

Prefer alpine versions (e.g. node:18-alpine). They are tiny — 5MB instead of 300MB. Faster to download and run.

3

Create a .dockerignore file

Like .gitignore, list files to exclude. Add node_modules, .git, .env to keep your image small and clean.

4

Never store secrets in Dockerfile

Do not write passwords or API keys in Dockerfile. Use environment variables (-e) or Docker secrets instead.

5

Clean up unused containers and images

Run `docker system prune` regularly to free disk space. They accumulate fast.

6

Use meaningful container names

Always use `--name mywebapp`. Random names make container management very confusing.

30 Docker Interview Questions

These are the most common Docker interview questions for beginners. Written in simple language — easy to read and remember.

Q1 What is Docker?

Docker is an open-source tool that packages applications and their dependencies into containers so they run consistently on any machine.

Q2 What is the difference between a Docker Image and Container?

An image is a blueprint (template). A container is a running instance of that image. One image can create many containers.

Q3 What is Docker Hub?

Docker Hub is the official cloud registry where you can find and download ready-made Docker images for free.

Q4 What is a Dockerfile?

A text file with instructions to build a custom Docker image — base image, files to copy, commands to run, and the startup command.

Q5 What is detached mode (-d)?

Detached mode runs the container in the background so your terminal stays free. Use: `docker run -d nginx`.

Q6 What is port mapping in Docker?

Port mapping connects a port on your host machine to a port inside the container. Example: `-p 8080:80` maps host 8080 to container 80.

Q7 What is a Docker Volume?

Persistent storage for containers. Data in a volume survives even if the container is stopped or deleted.

Q8 What is the difference between Docker and a Virtual Machine?

Docker containers share the host OS kernel, start in seconds, and use very little memory. VMs have a full OS, take minutes to start, and use much more resources.

Q9 How do you check running containers?

Use `docker ps` for running containers. Use `docker ps -a` to see all containers including stopped ones.

Q10 How do you remove a container?

Stop it first: `docker stop <id>`, then delete: `docker rm <id>`. You cannot remove a running container without the `-f` flag.

Q11 What is docker-compose?

A tool to define and run multi-container apps using a `docker-compose.yml` file. Start everything with one command: `docker-compose up`.

Q12 What is a bind mount?

A bind mount links a folder from your host machine directly into the container. Changes on either side are immediately visible — great for development.

Q13 What is image layering?

Each Dockerfile instruction creates a new layer. Docker caches layers so if only the last step changes, only that layer rebuilds — making builds faster.

Q14 What happens when a container stops?

The container stops but is not deleted. Data written inside (not in a volume) is lost. Restart with `docker start <id>`.

Q15 What is the CMD instruction in Dockerfile?

CMD defines the default command that runs when the container starts. Example: `CMD ["node", "app.js"]`. It can be overridden when running the container.

Q16 What is the difference between CMD and ENTRYPOINT?

CMD provides default commands that can be overridden. ENTRYPOINT sets the main command that always runs and is harder to override. ENTRYPOINT is stricter.

Q17 What is docker exec?

`docker exec` runs a command inside an already running container. Use `docker exec -it <id> bash` to open an interactive terminal inside.

Q18 What does docker inspect do?

Shows detailed JSON info about a container or image — IP address, volumes, environment variables, network settings and more.

Interview Questions + Learning Roadmap

Q19 What is a Docker Registry?

A registry is a storage system for Docker images. Docker Hub is the public registry. Companies often set up private registries for internal images.

Q20 What is containerization?

Packaging an application with all its dependencies into a self-contained unit (container) so it runs consistently in any environment.

Q21 What is the EXPOSE instruction in Dockerfile?

EXPOSE tells Docker which port the container will use. It does not publish the port — you still need `-p` when running. It is documentation for image users.

Q22 How do you pass environment variables to a container?

Use the `-e` flag: `docker run -e DATABASE_URL=mydb myapp`. Or use `--env-file .env` to pass all variables from a file at once.

Q23 What is docker system prune?

Removes all stopped containers, unused images, unused networks and build cache. Use `-a` flag to also remove all unused images and free more space.

Q24 What is the FROM instruction in Dockerfile?

FROM defines the base image. Every Dockerfile must start with FROM. Example: `FROM python:3.11-slim` starts from official slim Python 3.11 image.

Q25 What is docker stats?

Shows live resource usage of running containers — CPU %, memory, network I/O, block I/O. Like Task Manager but for Docker containers.

Q26 What is the WORKDIR instruction?

Sets the working directory inside the container for all following instructions. Like `cd` into a folder. Example: `WORKDIR /app`

Q27 What is the difference between docker stop and docker kill?

`docker stop` sends a graceful shutdown and waits for the process to finish. `docker kill` immediately terminates it. Always prefer stop over kill.

Q28 What is docker logs?

`docker logs <id>` shows all output from a container — print statements, errors, server messages. Use `-f` to follow logs in real time.

Q29 What is a multi-stage build?

Uses multiple FROM statements in one Dockerfile. Build in one stage, copy only the final output to a small image. Result: much smaller production images.

Q30 What is the difference between docker rm and docker rmi?

`docker rm` removes a stopped container. `docker rmi` removes a Docker image. You must stop and remove all containers using an image before removing the image.

Docker Learning Roadmap

1

Install Docker & Run First Container

`docker run hello-world` — your first success 🎉

2

Learn Core Commands

`pull`, `run`, `ps`, `stop`, `rm`, `logs`, `exec` — practice every day

3

Write Your First Dockerfile

Build a custom image for a Node.js or Python app

4

Learn Docker Compose

Run multiple containers together — app + database

5

Push Image to Docker Hub

Share your image with the world — `docker push myimage`

☀️ **You are now ready.** Practice these commands daily. Build real projects with Docker. The more you use it, the more natural it feels. **Save this guide for future reference.**